

Linear regression

Phuong Luu Vo

September 15, 2026

1 Example

Assume that the number of dengue cases and the rainfall in the last 3 years are shown in Figure 1.

$$\begin{aligned} X^o = & [55, 102, 169, 177, 185, 236, 252, 269, 359, 380, 386, 115, \\ & 11, 16, 137, 149, 216, 234, 251, 253, 356, 364, 372, 362, \\ & 36, 58, 63, 77, 282, 287, 304, 305, 370, 374, 399, 279] \\ y = & [163, 239, 227, 364, 265, 354, 325, 364, 577, 529, 544, 214, \\ & 109, 9, 155, 179, 339, 434, 410, 373, 546, 486, 536, 471, \\ & 189, 275, 51, 184, 293, 501, 431, 462, 457, 514, 486, 461] \end{aligned}$$

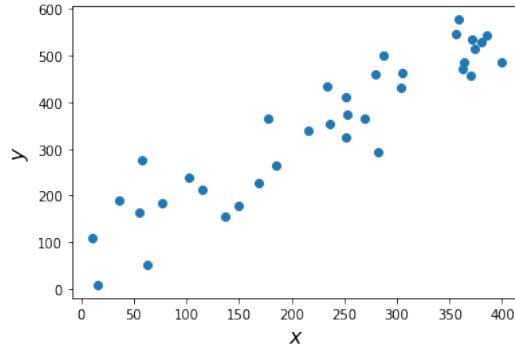


Figure 1: Dengue cases and the rainfall.

In case X^o has more than one feature, X^o is a matrix $X^o = \begin{bmatrix} (x_1^o)^T \\ \dots \\ (x_N^o)^T \end{bmatrix}$, in which x_i^o is the i observation and N is the number of observations.
We will find a linear function that fits the data.

2 Linear regression as convex optimization problem

Assuming the linear regression model is given by

$$w_0 + x_i^o w_1 = x_i^T w,$$

where $x_i = [1, x_i^o]^T$.

Error (residual) between the predicted value and the true value is:

$$L_i(w) = (y_i - x_i^T w)^2.$$

Total error is given by:

$$\begin{aligned} L(w) &= \sum_{i=1}^N L_i = \sum_{i=1}^N (y_i - x_i^T w)^2 \\ &= \|y - Xw\|_2^2. \end{aligned}$$

If using ℓ_1 norm, the total error is given by:

$$\begin{aligned} L(w) &= \sum_{i=1}^N |y_i - x_i^T w| \\ &= \|y - Xw\|_1. \end{aligned}$$

In general,

$$L(w) = \|y - Xw\|,$$

where $X = \begin{bmatrix} x_1^T \\ \dots \\ x_N^T \end{bmatrix}$

We find w to minimize $L(w)$

$$\min. L(w) = \|y - Xw\|. \quad (1)$$

This is a convex problem.

If using ℓ_2 -norm, it becomes a *least square* problem:

$$\min. \|y - Xw\|_2^2 = (y - Xw)^T (y - Xw). \quad (2)$$

First order condition is as follow:

$$\nabla_w (y - Xw)^T (y - Xw) = 2(X^T Xw - X^T y) = 0$$

Hence, the closed-form solution is given by

$$w^* = (X^T X)^{-1} X^T y. \quad (3)$$

3 Using CVXPY to solve the linear regression problem

Import:

```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt
```

Import dataset:

```
X = np.array((55,102,169,177,185,236,252,269,359,380,386,115, 11,16,137,149,216,
              234,251,253,356,364,372,362, 36,58,63,77,282,287,304,305,370,374,
```

```
Y = np.array((163,239,227,364,265,354,325,364,577,529,544,214, 109,9,155,179,339
              434,410,373,546,486,536,471,189,275,51,184,293,501,431,462,457,51
```

Add 1 column to X

```
X0 = np.ones((np.shape(X)[0]))
X_new = np.column_stack((X0,X))
```

Objective function:

```
def obj_fn(X, Y, beta):
    return cp.pnorm(X @ weight - Y, p=2)**2
```

Define the problem and solve:

```
weight = cp.Variable(np.shape(Xnew)[1])
problem = cp.Problem(cp.Minimize(obj_fn(X_new, Y, weight)))
problem.solve()
print(weight.value)
```

Solution w^* is

```
array([79.96960019, 1.16955022])
```

Figure 2 shows the fitted line.

```
Y_est=X.dot(weight.value[1]) + weight.value[0]
plt.scatter(X, Y)
plt.plot(X,Y_est, 'r')
plt.xlabel("x", fontsize=16)
plt.ylabel("y", fontsize=16)
```

4 Ridge Regression and Lasso Regression

Understand the following contents:

- Least-square https://www.cvxpy.org/examples/basic/least_squares.html

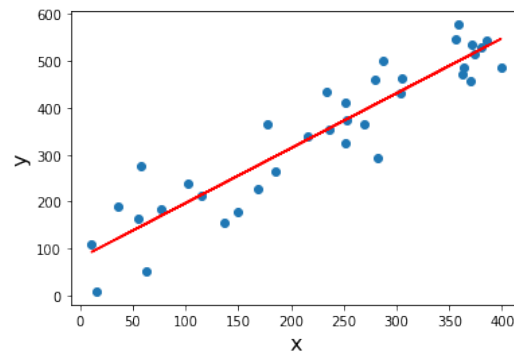


Figure 2: The fitted line.

- Ridge Regression, https://www.cvxpy.org/examples/machine_learning/ridge_regression.html
- Lasso regression https://www.cvxpy.org/examples/machine_learning/lasso_regression.html

References

- [1] Stephen P. Boyd and Lieven Vandenberghe. Convex optimization. Cambridge university press, 2004.

Binary classification - Logistic regression

Phuong Luu Vo

September 15, 2026

1 Sigmoid function

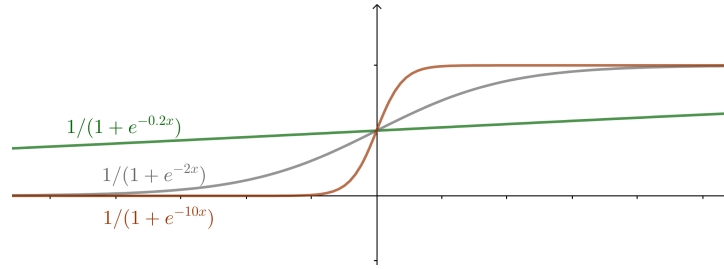


Figure 1: Sigmoid function $\frac{1}{1+e^{-ax}}$ with $a = 0.2, 2, 10$.

Sigmoid function is defined as follow:

$$\sigma(a) = \frac{1}{1 + e^{-a}}.$$

Some properties of sigmoid function:

- $0 < \sigma(a) < 1$, hence, we apply sigmoid function to model a probability.
- $\sigma(-a) = 1 - \sigma(a)$.
- Inverse function is called *logit*,

$$a = \log\left(\frac{\sigma}{1 - \sigma}\right)$$

- Derivative

$$\frac{d\sigma}{da} = \sigma(1 - \sigma).$$

2 Logistic regression

Given dataset (X, y) , in which $X = \begin{bmatrix} -x_1^T & - \\ \dots & \\ -x_N^T & - \end{bmatrix}$ is the matrix with N observations,

$y = \begin{bmatrix} y_1^T \\ \dots \\ y_N^T \end{bmatrix}$, where y_i is 0 or 1.

We use sigmoid function with parameter w to model the probability of an observation (x_i, y_i) :

$$P(y_i = 1|x_i) = \pi_i = \sigma(w^T x_i). \quad (1)$$

Hence,

$$P(y_i = 0|x) = 1 - \pi_i. \quad (2)$$

We rewrite the likelihood for observation x_i, y_i as follows:

$$P(y_i|x_i) = \pi_i^{y_i} (1 - \pi_i)^{1-y_i}. \quad (3)$$

Hence, the likelihood is:

$$P(y|X) = \prod_{i=1}^N \sigma(w^T x_i)^{y_i} (1 - \sigma(w^T x_i))^{1-y_i}. \quad (4)$$

We need to find w to maximize likelihood $P(y|X; w)$ (maximum likelihood estimation).

Logarithm transforms a product to a summation. The **log-likelihood function** is given by

$$\begin{aligned} \log(P(y|X)) &= \sum_{i=1}^N y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i) = - \sum_{i=1}^N BCE(y_i, \pi_i) \\ &= \sum_{i=1}^N y_i w^T x_i + \log(1 - \sigma(w^T x_i)) \\ &= \sum_{i=1}^N y_i w^T x_i - \log(1 + e^{w^T x_i}). \end{aligned}$$

where $BCE(\dots, \dots)$ is the cross-entropy between two Bernoulli distributions.

Hence, the optimization problem of logistic regression is:

$$\max. \sum_{i=1}^N y_i w^T x_i - \log(1 + e^{w^T x_i}) \quad (5)$$

$$\text{or } \min. \sum_{i=1}^N BCE(y_i, \pi_i). \quad (6)$$

3 Example of logistic regression in CVXPY

The example with artificial dataset.

```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt

    Create 100 data points for training and 100 others for testing.  $x_i$  has 02
    features,  $y_i$  is 0 or 1.

np.random.seed(1)
d = 2      # number of features
m = 100    # number of samples

weight_true = (np.random.random(d) - 0.5)*2
X = (np.random.random((m, d)) - 0.5)*5
Y = np.rint(sigmoid(X @ weight_true + np.random.randn(m)*0.5))

m_test=m
X_test = (np.random.random((m_test, d)) - 0.5)*5
Y_test = np.rint(sigmoid(X_test @ weight_true + np.random.randn(m_test)*0.5))

    Verify the shape of X and Y:
print(X.shape, Y.shape)

(100, 2) (100,)

    We can plot the data points on 2D:

plt.scatter(X[Y == 0, 0], X[Y == 0, 1], marker='o')
plt.scatter(X[Y == 1, 0], X[Y == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("train_dataset.png")

    Input the problem to CVX

weight = cp.Variable(d)
weight0 = cp.Variable()
log_likelihood = cp.sum(
    cp.multiply(Y, X @ weight + weight0) - cp.logistic(X @ weight + weight0)
)

problem = cp.Problem(cp.Maximize(log_likelihood))
problem.solve()

print(weight0.value)
print(weight.value)
```

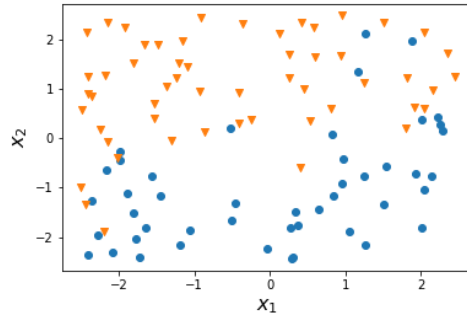


Figure 2: Dataset in 2D.

The optimal weight is:

```
0.03178046482109968
[-0.73230372  1.93465795]
```

Calculate error:

```
def error(scores , labels):
def error(scores , labels):
    scores[scores >= 0.5] = 1
    scores[scores < 0.5] = 0
    return scores , np.sum(np.abs(scores - labels)) / float(np.size(labels))
```

```
_, error_train = error( sigmoid(X @ weight.value + weight0.value), Y)
Y_predict, error_test = error( sigmoid(X_test @ weight.value + weight0.value), Y)
print("Train error:-", error_train)
print ("Test error:-", error_test)
```

Results:

```
Train error:  0.14
Test error:   0.17
```

Plot in 2D

```
plt.scatter(X_test[Y_test == 0, 0], X_test[Y_test == 0, 1], marker='o')
plt.scatter(X_test[Y_test == 1, 0], X_test[Y_test == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("test_dataset.png")

plt.scatter(X_test[Y_predict == 0, 0], X_test[Y_predict == 0, 1], marker='o')
plt.scatter(X_test[Y_predict == 1, 0], X_test[Y_predict == 1, 1], marker='v')
plt.xlabel("$x_1$", fontsize=16)
plt.ylabel("$x_2$", fontsize=16)
plt.savefig("prediction.png")
```

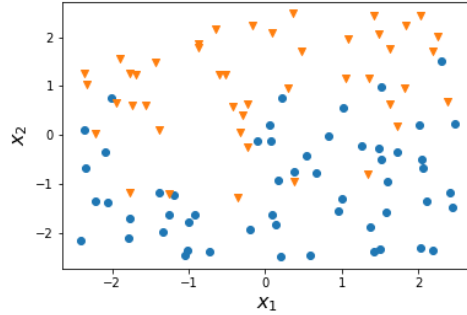


Figure 3: Test set.

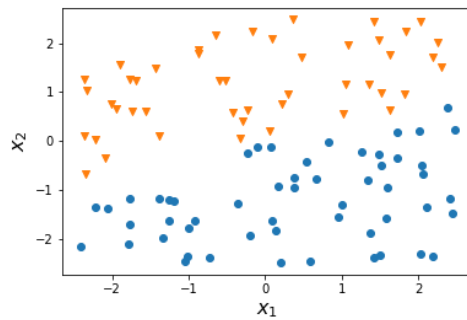


Figure 4: Results of classification by logistic regression model.

4 Logistic regression with ℓ_1 norm regularization

Understand and run the code in this page: https://www.cvxpy.org/examples/machine_learning/logistic_regression.html.

References

- [1] Stephen P. Boyd, and Lieven Vandenberghe. Convex optimization. Cambridge university press, 2004.
- [2] Logistic regression with ℓ_1 norm regularization. https://www.cvxpy.org/examples/machine_learning/logistic_regression.html.

Multiclass classification - Softmax regression

Phuong Luu Vo

September 21, 2026

1 K -class classification with softmax

Given a dataset consisting of (N) observations for a multi-class classification task:

- Let $x_i \in \mathbb{R}^P, i = 1, \dots, N$ denote the input vector for observation (i) . (If the first element of (x_i) is the intercept term, then $x_i = [1, x_{i1}, \dots, x_{i,P-1}]^T$.)
- Suppose there are K classes,

$$\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K.$$

- The class label of observation (i) is represented using a **one-hot encoded** vector y_i . If observation i belongs to class $\mathcal{C}_k$, then:

$$y_i = \begin{bmatrix} y_{i1} \\ \dots \\ y_{ik} \\ \dots \\ y_{iK} \end{bmatrix} = \begin{bmatrix} 0 \\ \dots \\ 1 \\ \dots \\ 0 \end{bmatrix},$$

$$y_{ik} = 1 \text{ and } y_{ij} = 0 \text{ for all } j \neq k. \quad y_{i1} + y_{i2} + y_{i3} + \dots + y_{iK} = 1.$$

We model the probability that a sample i with feature vector x_i belongs to class k using softmax function as follows:

$$p(y_i \in \mathcal{C}_k | x_i; W) = \pi_{ik} = \frac{e^{w_k^T x_i}}{\sum_{k'=1}^K e^{w_{k'}^T x_i}}.$$

Softmax function is a general form of logistic regression.

Define the weight matrix W :

$$W = \begin{bmatrix} w_1^T \\ \dots \\ w_K^T \end{bmatrix}, \tag{1}$$

in which $w_k \in \mathbb{R}^P$ is the weight vector associated with class k .

Likelihood of observation x_i, y_i is given by:

$$p(y_i | x_i; W) = \prod_{k=1}^K (\pi_{ik})^{y_{ik}}. \quad (2)$$

Hence, the likelihood for all observations is given by:

$$p(y|X; W) = \prod_{i=1}^N p(y_i|x_i) \quad (3)$$

$$= \prod_{i=1}^N \prod_{k=1}^K (\pi_{ik})^{y_{ik}}. \quad (4)$$

Maximum likelihood problem: Finding optimal W such that maximizing $p(y|X; W)$.

Logarithm function transforms a multiplication to an addition, which is easier to optimize. **Maximum log-likelihood problem** is given by:

$$\max. \sum_{i=1}^N \sum_{k=1}^K \log((\pi_{ik})^{y_{ik}}) \quad (5)$$

$$\sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \pi_{ik} \quad (6)$$

$$= - \sum_{i=1}^N CE(y_i, \pi_i) \quad (7)$$

where $CE(.,.)$ is the cross-entropy between y_i and $\pi_i = [\pi_{i1}, \dots, \pi_{iK}]^T$.

The above problem is a convex.

2 Example

Students may generate an artificial dataset and use the CVXPY library to obtain the optimal solution to the softmax problem (see this reference <https://machinelearningcoban.com/2017/02/17/softmax/> for example on artificial dataset generation).